

Objectives

- Explain about Angular Forms
- Explain Reactive Forms
- Explain Form model setup in Reactive forms

In this hands-on lab, you will learn how to:

- Work with Reactive Forms in Angular

Prerequisites

The following is required to complete this hands-on lab:

- Visual Studio Code editor
- NodeJS
- Angular CLI

Notes

Estimated time to complete this lab: **60 minutes**.

Reactive Form - New Component and Name field inclusion

Similar to the hands on done for Template Driven Form, we will develop the same edit employee form using Reactive Forms. The form will be gradually built with increasing complexity.

In this hands-on, we will create a new component and add the name field for employee.

Follow instructions below:

- Create a new component named 'edit-emp-reactive'
- Include ReactiveFormsModule in app.module.ts
- Import FormControl in edit-emp-reactive component.ts.
- Include a new instance variable in component.ts. Refer code below

```
name = new FormControl("");
```

- Refer below code to include the form element and map it to the form control

```
Name: <input type="text" [formControl]="name">
```

- Set a default value for the field by changing the name property as specified below.

```
name = new FormControl('John');
```

Sample Layout

Angular Learning [View Employee](#)

Edit Employee

Name

Reactive Form - Include form group and validation for name field

Instructions for getting employee data from instance variable and include validations:

- Import Employee, Skill and Department into component.ts
- Copy and paste the Employee test data from template driven form instance variable
- Set the data for form control from employee property. Refer code below.

```
name = new FormControl(this.employee.name);
```

- Include FormGroup and Validators in import

```
import { FormGroup, FormControl, Validators } from '@angular/forms';
```

- Define form group property in component.ts as specified below:

```
this.empForm = new FormGroup({  
  'name': new FormControl(this.employee.name, [  
    Validators.required,  
    Validators.minLength(4),  
    Validators.maxLength(20)  
  ])  
});
```

- Include getter method for name. This is for the validator to retrieve the value in the form field.

```
get name() { return this.empForm.get('name'); }
```

- Modify template with inclusion of FormGroup and use the template driven form validation code.

Reactive Form - Include salary, permanent and department fields

Implement rest of the form fields for edit employee form. Refer information below

- Utilize the HTML template code implemented in template driven form including display of employee JSON and form JSON
- Define department as FormControl
- Define the department drop down in HTML template as specified below:

```
<div>
  <h3>Department</h3>
  <select formControlName="department">
    <option *ngFor="let department of departments"
      [ngValue]="department"
      [selected]="department.id === employee.department.id">
      {{department.name}}
    </option>
  </select>
</div>
```

- Include additional getter methods for each field
- Implement onSubmit() method to print the form details in console by using the value property of form group

Reactive Form - Using FormBuilder

Implement employee edit using FormBuilder. Refer instructions below:

- Create a new component named 'edit-emp-form-builder'
- Copy existing html and ts code from reactive form
- Modify FormControl definition to use FormBuilder
- Inject form builder in constructor